FOOD ORDERING MANAGEMENT SYSTEM

DBMS PROJECT REPORT

1. INTRODUCTION

The Food Ordering Management System is designed to organise information associated with customers, orders, products and payments in a relational database. A food order can contain one or more products, and each order is associated with a customer. Payment records are linked to orders so that payment mode, payment date and payment status can be reviewed.

The provided SQL queries focus mainly on retrieving and analysing data rather than on user-interface functionality. They combine related tables to display complete order details and generate reports for customer purchase history, order counts, sales revenue and payment analysis.

2. OBJECTIVES

- To retrieve order information together with related payment details.
- To display customer, order, product, quantity and price information in one result.
- To list all customers, including customers who have not placed orders.
- To identify customers without orders and products without order-detail records.
- To display order and payment records using outer joins.
- To retrieve a customer's purchase history.
- To calculate total spending and the number of orders for each customer.
- To find each customer's latest purchase.
- To prepare product sales, revenue and payment-mode summaries.
- To identify customers with the maximum number of orders or highest spending.

3. SYSTEM OVERVIEW

Main Database Areas

Area	Purpose based on the queries
Customer	Provides CUSTOMER_ID, CUSTOMER_NAME, EMAIL and PHONE for customer reports.
Orders	Provides ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT and ORDER_STATUS.

Order details	Provides the product and quantity associated with an order, including QUANTITY and PRICE.
Product	Provides PRODUCT_ID, PRODUCT_NAME and PRICE.
Payment	Provides PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS and transaction amount.

DATABASE TABLES USED

The query document references the following tables and columns. The exact data types and constraints cannot be confirmed from SELECT statements alone.

Table	Columns referenced in the supplied queries
CUSTOMER	CUSTOMER_ID, CUSTOMER_NAME, EMAIL, PHONE
ORDERS	ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS
ORDER_DETAILS	ORDER_ID, PRODUCT_ID, QUANTITY, PRICE
PRODUCT	PRODUCT_ID, PRODUCT_NAME, PRICE
PAYMENT	PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT

The query file does not provide CREATE TABLE definitions, primary-key declarations, foreign-key declarations, sample INSERT statements or actual output screenshots. Add those details from your implementation if your report format requires them.

4. SQL CONCEPTS APPLIED

1 INNER JOIN

2 LEFT JOIN

3 RIGHT JOIN

4 Aggregate Functions

5 GROUP BY, HAVING and Subqueries

6 CASE Expression

5. QUERY IMPLEMENTATION

1. Implement INNER JOIN

- **Display customer details along with their orders..**

```
SQL> SELECT C.CUSTOMER_NAME,O.ORDER_ID,O.TOTAL_AMOUNT FROM CUSTOMER C INNER JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID;

CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_ID   TOTAL_AMOUNT
---------- ------------
SNEHA
106                550

VIJAY KUMAR
107               1250

ANITHA
108                480


CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_ID   TOTAL_AMOUNT
---------- ------------
SURESH
109                890

KAMESH
110                900
```

- **Display order details with payment information.**

```
SQL> SELECT O.ORDER_ID, O.TOTAL_AMOUNT, O.ORDER_DATE, O.CUSTOMER_ID, P.PAYMENT_DATE, P.PAYMENT_MODE, P.PAYMENT_STATUS FROM PAYMENT P JOIN ORDERS O ON P.ORDE
R_ID = O.ORDER_ID;

ORDER_ID   TOTAL_AMOUNT ORDER_DAT CUSTOMER_ID PAYMENT_D PAYMENT_MODE
---------- ------------ --------- ----------- --------- --------------------
PAYMENT_ST
----------
106                550 23-SEP-26         101 24-SEP-26 UPI
COMPLETED

107               1250 15-MAY-26         102 15-MAY-26 CASH
PENDING

108                480 09-MAR-26         103 09-MAR-26 NET BANKING
FAILED


ORDER_ID   TOTAL_AMOUNT ORDER_DAT CUSTOMER_ID PAYMENT_D PAYMENT_MODE
---------- ------------ --------- ----------- --------- --------------------
PAYMENT_ST
----------
109                890 10-MAY-26         104 10-MAY-26 CARD
PENDING

110                900 10-OCT-26         105 11-OCT-26 NET BANKING
COMPLETED
```

- **Retrieve products purchased by customers.**

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, O.ORDER_ID, P.PRODUCT_ID, P.PRODUCT_NAME, OD.QUANTITY, OD.PRICE, OD.QUANTITY* OD.PRICE AS TOTAL_AMOUNT FROM CUST
OMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON OD.PRODUCT_ID = P.PRODUCT_ID ORDER
 BY C.CUSTOMER_ID, O.ORDER_DATE;

 CUSTOMER_ID CUSTOMER_NAME       ORDER_ID   PRODUCT_ID PRODUCT_NAME        QUANTITY      PRICE TOTAL_AMOUNT
------------ ------------------- ---------- ---------- ------------------- -------- ---------- ------------
         101 SNEHA               106        P01        IDLI                       2     250.00       500.00
         102 VIJAY KUMAR         107        P02        BUTTER NAAN                3     240.00       720.00
         103 ANITHA              108        P03        CHICKEN FRIED RICE         5     250.00      1250.00
         104 SURESH              109        P05        BROWNIE                    2     300.00       600.00
         105 KAMESH              110        P06        CHICKEN BURGER             4     400.00      1600.00
```

Implement LEFT JOIN

- **Display all customers including customers who have not placed orders.**

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, C.EMAIL, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOM
ER_ID = O.CUSTOMER_ID ORDER BY C.CUSTOMER_ID;

 CUSTOMER_ID CUSTOMER_NAME       EMAIL                                          ORDER_ID   ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------------ ------------------- ---------- ---------- --------- ---------- ---------
------------ -------------------
         101 SNEHA               sneha@gmail.com                                106        23-SEP-26      550.00 Pending
         102 VIJAY KUMAR         Vijay@gmail.com                                107        15-MAY-26     1250.00 Delivered
         103 ANITHA              anitha@gmail.com                               108        09-MAR-26      480.00 Preparing
         104 SURESH              suresh@gmail.com                               109        10-MAY-26      890.00 Preparing
         105 KAMESH              kamesh@gmail.com                               110        10-OCT-26      900.00 Delivered
```

- **Find customer without any purchases;**

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, C.EMAIL, C.PHONE FROM CUSTOMER C
 LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID WHERE O.ORDER_ID IS NUL
L ORDER BY C.CUSTOMER_ID;

no rows selected
```

- **Display all product including products with no sales;**

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRICE FROM PRODUCT P LEFT JOIN
  2  ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID WHERE OD.PRODUCT_ID IS NULL ORDER BY P.PRODUCT_ID;

no rows selected
```

Implement RIGHT JOIN:

- **Display all order with customer information:**

```
SQL> SELECT O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS, C.CUSTOMER_ID, C.CUSTOMER_NAME, C.EMAIL FROM CUSTOMER C RIGHT JOIN ORDERS O ON C.CUSTO
MER_ID = O.CUSTOMER_ID ORDER BY O.ORDER_DATE;

ORDER_ID   ORDER_DAT TOTAL_AMOUNT ORDER_STATUS         CUSTOMER_ID CUSTOMER_NAME    EMAIL
---------- --------- ------------ ------------------ ----------- ------------------ --------------------------------------------------------------------
--------------------------------
108        09-MAR-26       480.00 Preparing                  103 ANITHA  anitha@gmail.com
109        10-MAY-26       890.00 Preparing                  104 SURESH  suresh@gmail.com
107        15-MAY-26      1250.00 Delivered                  102 VIJAY KUMAR     Vijay@gmail.com
106        23-SEP-26       550.00 Pending                    101 SNEHA   sneha@gmail.com
110        10-OCT-26       900.00 Delivered                  105 KAMESH  kamesh@gmail.com
```

- **Find orders where customer details are missing:**

```
SQL> SELECT O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM CU
STOMER C RIGHT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID WHERE C.CUSTOM
ER_ID IS NULL;

no rows selected
```

.

- **Display all payment record with order details:**

```
SQL> SELECT P.PAYMENT_ID, P.PAYMENT_DATE, P.PAYMENT_MODE, P.PAYMENT_STATUS, P.TRANSACTION_AMOUNT, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT AS ORDER_AMOUNT,
O.ORDER_STATUS FROM ORDERS O RIGHT JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID ORDER BY P.PAYMENT_DATE;

PAYMENT PAYMENT_D PAYMENT_MODE       PAYMENT_ST TRANSACTION_AMOUNT ORDER_ID
------- --------- ------------------ ---------- ----------------- ----------
ORDER_DAT ORDER_AMOUNT ORDER_STATUS
--------- ------------ ------------------
PAY103  09-MAR-26 NET BANKING        FAILED                 1000 108
09-MAR-26          480 Preparing

PAY104  10-MAY-26 CARD               PENDING                2000 109
10-MAY-26          890 Preparing

PAY102  15-MAY-26 CASH               PENDING                 900 107
15-MAY-26         1250 Delivered


PAYMENT PAYMENT_D PAYMENT_MODE       PAYMENT_ST TRANSACTION_AMOUNT ORDER_ID
------- --------- ------------------ ---------- ----------------- ----------
ORDER_DAT ORDER_AMOUNT ORDER_STATUS
--------- ------------ ------------------
PAY101  24-SEP-26 UPI                COMPLETED              3000 106
23-SEP-26          550 Pending

PAY105  11-OCT-26 NET BANKING        COMPLETED              1500 110
10-OCT-26          900 Delivered
```

Retrieve complete order details Generate a complete order report by combining: ● Customer Name ● Product Name ● Quantity ● Order Date ● Total Amount ● Payment Status

```
SQL> SELECT C.CUSTOMER_NAME, P.PRODUCT_NAME, OD.QUANTITY, O.ORDER_DATE, O.TOTAL_AMOUNT, PM.PAYMENT_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O
.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON OD.PRODUCT_ID = P.PRODUCT_ID LEFT JOIN PAYMENT PM ON O.ORDER_ID = PM.ORDER_
ID ORDER BY O.ORDER_DATE, C.CUSTOMER_ID;

CUSTOMER_NAME
--------------------------------------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_ST
---------- --------- ------------ ----------
ANITHA
CHICKEN FRIED RICE
         5 09-MAR-26          480 FAILED

SURESH
BROWNIE
         2 10-MAY-26          890 PENDING

CUSTOMER_NAME
--------------------------------------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_ST
---------- --------- ------------ ----------

VIJAY KUMAR
BUTTER NAAN
         3 15-MAY-26         1250 PENDING

SNEHA
IDLI

CUSTOMER_NAME
--------------------------------------------------------------------------------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY ORDER_DAT TOTAL_AMOUNT PAYMENT_ST
---------- --------- ------------ ----------
         2 23-SEP-26          550 COMPLETED

KAMESH
CHICKEN BURGER
         4 10-OCT-26          900 COMPLETED
```

Display customer purchase history:

- **All products purchased by a customer:**

```
SQL> SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, P.PRODUCT_NAME, OD.QUANTITY, OD.PRICE FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON OD.PRODUCT_ID = P.PRODUCT_ID WHERE C.CUSTOMER_ID = '105' ORDER BY O.ORDER_DATE;

CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_ID   ORDER_DAT
---------- ---------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY      PRICE
---------- ----------
KAMESH
110        10-OCT-26
CHICKEN BURGER
         4        400
```

- **Total amount spent by each customer:**

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, ROUND(SUM(O.TOTAL_AMOUNT), 2) AS TOTAL_AMOUNT_SPENT FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_
ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_AMOUNT_SPENT DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_AMOUNT_SPENT
------------------
        102
VIJAY KUMAR
              1250

        105
KAMESH
               900

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_AMOUNT_SPENT
------------------

        104
SURESH
               890

        101
SNEHA

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_AMOUNT_SPENT
------------------
               550

        103
ANITHA
               480
```

- **Number of orders placed by each customer.**

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP
BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY NUMBER_OF_ORDERS DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
---------------
       105
KAMESH
              1

       102
VIJAY KUMAR
              1

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
---------------

       101
SNEHA
              1

       103
ANITHA

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
---------------
              1

       104
SURESH
              1
```

Latest purchase detail of customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE AS LATEST_PURCHASE_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM CUSTOMER C JOIN ORDERS O O
N C.CUSTOMER_ID = O.CUSTOMER_ID WHERE O.ORDER_DATE = (SELECT MAX(O2.ORDER_DATE) FROM ORDERS O2 WHERE O2.CUSTOMER_ID = C.CUSTOMER_ID ) ORDER BY CUSTOMER_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_ID   LATEST_PU TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
        101
SNEHA
106        23-SEP-26          550 Pending

        102
VIJAY KUMAR
107        15-MAY-26         1250 Delivered

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_ID   LATEST_PU TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------

        103
ANITHA
108        09-MAR-26          480 Preparing

        104
SURESH

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_ID   LATEST_PU TOTAL_AMOUNT ORDER_STATUS
---------- --------- ------------ --------------------
109        10-MAY-26          890 Preparing

        105
KAMESH
110        10-OCT-26          900 Delivered
```

Generate Multi-Table Business Reports

Report 1: Customer Order Report Display: ● Customer name ● Order ID ● Order date ● Order status

```
SQL> SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.ORDER_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID ORDER BY O.ORDER_DATE;

CUSTOMER_NAME
--------------------------------------------------------------------------------
ORDER_ID   ORDER_DAT ORDER_STATUS
---------- --------- --------------------
ANITHA
108        09-MAR-26 Preparing

SURESH
109        10-MAY-26 Preparing

VIJAY KUMAR
107        15-MAY-26 Delivered
```

Report 2: Sales Report Display: ● Product name ● Quantity sold ● Total revenue

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS QUANTITY_SOLD, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUC
T_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTAL_REVENUE DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
CHICKEN BURGER
            4          1600

CHICKEN FRIED RICE
            5          1250

BUTTER NAAN
            3           720


PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD TOTAL_REVENUE
------------- -------------
BROWNIE
            2           600

IDLI
            2           500
```

Report 3: Payment Analysis Report Display: ● Payment mode ● Number of transactions ● Successful payments

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS NUMBER_OF_TRANSACTIONS, SUM( CASE WHEN
 PAYMENT_STATUS = 'COMPLETED' THEN 1 ELSE 0 END ) AS SUCCESSFUL_PAYMENTS
FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY NUMBER_OF_TRANSACTIONS DESC;

PAYMENT_MODE         NUMBER_OF_TRANSACTIONS SUCCESSFUL_PAYMENTS
-------------------- ---------------------- ------------------
NET BANKING                               2                  1
CARD                                      1                  0
UPI                                       1                  1
CASH                                      1                  0
```

Report 4: Customer Purchase Analysis Find: Top purchasing customers

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C
CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENDING
--------------
       102
VIJAY KUMAR
         1250

       105
KAMESH
          900

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENDING
--------------

       104
SURESH
          890

       101
SNEHA

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENDING
--------------
          550

       103
ANITHA
          480
```

- **Customer with maximum orders:**

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C
CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(*) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
---------------
       105
KAMESH
              1

       102
VIJAY KUMAR
              1

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
---------------

       104
SURESH
              1

       103
ANITHA

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS
---------------
              1

       101
SNEHA
              1
```

- **Customers with highest spending**

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPE
NDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP B
Y C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) =
  2  (SELECT MAX(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPE
NDING FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
----
TOTAL_SPENDING
--------------
        102
VIJAY KUMAR
          1250
```

12. CONCLUSION

The Food Ordering Management System project demonstrates the use of SQL to retrieve and analyse data from CUSTOMER, ORDERS, ORDER_DETAILS, PRODUCT and PAYMENT. INNER JOIN combines matching customer, order and product information, while LEFT JOIN and RIGHT JOIN help retain records that may not have matches in related tables. Aggregate functions and subqueries generate reports on customer spending, order counts, product sales, payment modes, latest purchases and top customers. These operations show how a relational database can support practical reporting and analysis for a food ordering workflow.

•